0 The big picture

Workload model (L03) → Performance model (L04–05) → Performance test (L07) → Diagnose: model vs measurement → Optimise the bottleneck ↻ test

Phase	Question	Tools
Measure	What is it doing now?	Profilers (`cProfile` , `py-spy`), monitors (`psutil`), load generators
Model	Where are the limits?	Little's Law, operational laws, M/M/1, Kingman, Erlang B
Test	Does reality match the model?	Load / stress / soak / regression / benchmark / pipe-clean
Optimise	Fix the *bottleneck*	Reduce V, reduce S, add capacity, cut variance
Deploy	Keep the gain	Resource limits, autoscaling, headroom

GOLDEN RULE · Optimise only when (1) the bottleneck is **measured**, not suspected, (2) the gain justifies the cost (Amdahl), and (3) the fix can be **verified with numbers** under the same workload.

1 Core metrics and laws

R = **W** (waiting) + **S** (service) · **λ** = arrival rate · **μ** = 1/S
X = throughput · **U** = utilisation (busy %) · **ρ** = λS · **N** = no. in system

Formula	Name	Use it when…	Example
R = W + S	Response time	Splitting any delay into waiting and working	Clinic: 25+15 min wait, 10+5 min service → R = 55 min, 73% wait
U = X·S	Utilisation Law	Always. U = λS only if stable (X = λ)	5 pies × 10 min in 60 min → 83%
ρ = λS = λ/μ	Traffic intensity	Stability check (ρ < 1)	λ = 3/s, S = 100 ms → 30%
μ = 1/S	Service rate	Converting S to capacity	S = 10 ms → 100 tps
λ_{max} = 1/S	Max stable arrival (1 server)	Quick capacity	S = 6 min → 10 patients/hr
N = X·R	Little's Law	Any stable system or subsystem	7 req/s × 0.1 s = 0.7 busy DB conns
N_q = X·W	Little (queue only)	Mean queue length	—
λ = arrivals / window	Arrival rate	Always use the **whole window**	4 in 20 s → 0.2/s (not 1/mean gap)
X = completions / window	Throughput	Supply side; X < λ ⇒ overload	—

W/S trick: service time = only the time the server is actively working on *your* request. Everything else (queueing, interruptions, waiting for results) is wait.

PERCENTILES, TAILS AND SLOS

Item	Formula / rule	When to use
Nearest-rank percentile (general)	k = ⌈(p/100)·n⌉ → k-th smallest of n	p95/p99 from a `.jtl`
Mean vs median vs p9x	Mean dragged up by outliers; p95/p99 expose the tail	Use **p95 or p99** for SLOs
SLI / SLO / SLA	Metric → target → contract with penalty	p99 (SLI) < 200 ms (SLO), penalty (SLA)
Fan-out tail	$P(\geq 1 \text{ slow}) = 1 - (1-q)^N$	q = 1%, N = 100 calls → **63%**
Apdex	(Satisfied + ½·Tolerating) / Total	JMeter: Sat ≤ T 500 ms, Tol ≤ F 1,500 ms
Perception	0.1 s instant · 1 s flow kept · 10 s progress bar	Setting R targets

2 Queueing models

ρ = how busy each server is (0–1) · **c** = servers · **W** = queue wait
Ca / Cs = variability of arrivals / service (C = σ/mean, σ = std dev)

Model	Key formulas	Exam cue → what to do
M/M/1	ρ = λS < 1 · W = ρ/(1−ρ)·S · R = S/(1−ρ) · N = ρ/(1−ρ)	**Cue:** 1 server, "random/Poisson" → plug into W, R, N directly
M/M/c	ρ = λS/c < 1 (wait needs Erlang C — conceptual)	**Cue:** 1 queue, c servers → ρ = λS/c < 1? c↓ ⇒ W↑↑
G/G/1 Kingman	$W \approx \rho/(1-\rho) \cdot (C_a^2+C_s^2)/2 \cdot S$	**Cue:** 1 server + Ca/Cs or σ given → M/M/1 wait × (Ca²+Cs²)/2
G/G/c Sakasegawa	$W \approx (C_a^2+C_s^2)/2 \cdot \rho^{\sqrt{(2(c+1))}-1} / (c(1-\rho)) \cdot S$	**Cue:** c servers + variability → explain trend only: W↑ if ρ↑ or C↑
Coeff. of variation	C = σ / mean	Square for Kingman · 0 reg · 1 Markov · >1 bursty

Hockey-stick: at low ρ, R ≈ S; as ρ → 1, R → ∞.

ρ	0.5	0.6	0.7	0.8	0.9	0.95
ρ/(1−ρ)	1.0	1.5	2.33	**4.0**	**9.0**	19.0

Scenario	Working	Answer
M/M/1, ρ = 0.8, S = 100 ms	R = 0.1 / 0.2	R = 500 ms (W = 400)
Kingman ρ=0.8, C_a^2=1, C_s^2=4, S=100 ms	4 × 2.5 × 100	W = **1,000 ms** (2.5× M/M/1)
Kingman ρ=0.9, C_s^2=9, S=50 ms	9 × 5 × 50	W = **2,250 ms** vs 450 if C_s^2=1
M/M/c λ=8, S=148 ms, c = 3 → 2	8×0.148/3 → /2	ρ 0.40 → 0.60; wait 7 → 58 ms

One fast vs many slow: same total capacity — **one fast server gives lower latency** (each job gets the full rate). Many servers win on fault tolerance, cost and scaling limits, not queueing.

3 Operational laws, bottlenecks, bounds

X = throughput · **k** = one device · **V** = visits per txn
S = time per visit · **D** = V×S · **U** = busy % · **c** = servers

Formula	Name	Use it when…	Example
$X_k = V_k \cdot X$	Forced Flow	Device load from system load	4 SQL/txn × 25 tps = **100 qps**
$D_k = V_k \cdot S_k$	Service demand	Total time one txn needs at k	4 × 5 ms = 20 ms
$U_k = X \cdot D_k$	Service Demand Law	Per-device utilisation	30 tps × 20 ms = 60%
$D_k = U_k / X$	Demand from monitoring	No visit tracing available	—
$X \leq 1/D_{max}$	Throughput ceiling (1 srv)	Max system throughput	D_{max} 36 ms (DB) → **27.8/s**
$X \leq c/D_k$	Ceiling (c servers)	Multi-server devices	Litho 3/8.6 h = 8.4 lots/day
$R \geq \Sigma D_k$	R lower bound	Is a target even feasible?	5+10+36+4+25 = **80 ms**
$\text{argmin}_k\, c_k/D_k$	Capacity bottleneck	What saturates first	**Lowest ceiling**, not largest D

Background load (L05): capacity C, background b, per-request demand D → **$\lambda_{max} = (C - b)/D$**; take the **minimum** across devices.

Device	D	b	Capacity	λ_{max}
Memory	10 KB	2 GB	8 GB	6 GB / 10 KB ≈ 600,000/s
Disk 1	20 MB	50 GB	500 GB	450 GB / 20 MB = 22,500/s
Disk 2	100 KB	500 GB	2 TB	1,500 GB / 100 KB = 15,000,000/s
CPU	0.5%	0%	100%	100 / 0.5 = **200/s ← bottleneck**

From the L05 table, 1 TB = 1,000 GB. Max system rate = minimum row: 200 req/s.

BOTTLENECK ≠ HOTSPOT · The *bottleneck* has the lowest ceiling; the *hotspot* is where queueing builds. They differ when **variability** drives the queue — wafer fab: λ = 3.05 vs ceiling 5.6, yet litho queue tripled.

L04 RUNNING CASE · LARGEST D ≠ BOTTLENECK

Tier	D	c	Ceiling c/D	U @ X = 8.06/s
API	148 ms (largest)	3	3 / 0.148 ≈ 20.3/s	0.40 per server
DB	78 ms	1	**1 / 0.078 ≈ 12.8/s ← bottleneck**	0.63
Storage	16 ms	2	2 / 0.016 = 125/s	0.06 per node

R ≥ ΣD = 148 + 78 + 16 = 242 ms. Target of 20 tps is infeasible without fixing the DB. API-02 down (c = 3→2): API ceiling 20.3 → 13.5/s, ρ 0.40 → 0.60, wait 7 → 58 ms; DB (12.8) still the bottleneck, just.
Per-day (×24!): fab, 1 tool each: X ≤ 24/8.6 ≈ **2.8 lots/day** (litho). 3 litho tools → 8.4/day, so deposition 24/6.7 ≈ **3.6/day** becomes the system ceiling.

SATURATED or just congested? See the box at the bottom-left of page 2.

4 Sizing formulas ("how many?")

c = servers needed · **Ut** = target utilisation · **L** = concurrent users
N = users · **T** = ramp-up · **W** = session length · **Z** = think time

Question	Formula	Example
Servers needed	**$c = \lceil \lambda S / U_{target} \rceil$**	2,000 req/s, 100 req/s each (S = 10 ms), 70% → ⌈20/0.7⌉ = **29**
Concurrent users (open)	**$L = \lambda \cdot W_{session}$**	20 users/s × 500 s = **10,000**
Concurrent (batch ramp)	**L = min((N/T)·W, N)**	100 users in 5 s, 10 s sessions → min(200,100) = **100**
JMeter ramp-up for L	**T = N·W / L**	N=200, W=5 s, L=100 → **10 s**; W=240 s → **480 s**
Closed-system load	**X ≈ N/(R+Z)**	200 users, Z = 10 s, R = 1.11 s → 18 tps
Peak vs average	peak = avg × peak factor	9.3/s × 3 = 28/s (e-filing)
Workload chain	pop × active% × actions ÷ window × S ÷ U_t	2 M × 20% = 400 k → 9.3/s → ×3 → ×30 req → 830 req/s → ≈ 24 servers
Soak time to failure	**t = free mem / slope**	16 GB ÷ 3.4 MB/min ≈ **3.3 days**
Regression threshold	**$\Delta = (p95_{new} - p95_{base}) / p95_{base}$**	0.46 → 0.66 s = **+45%** (flag if > 10%)

5 Interactive (closed) systems

N = X(R + Z) ⇒ R = N/X − Z

N = number of active users
Z = avg think time: pause between getting a response and submitting the next request
Each user is always **thinking (Z)** or **waiting (R)**

- **Use when** reading load-test output (tool fixes N and Z, measures X).
- N = 200, Z = 10 s, X = 18 tps → R = 11.11 − 10 = **1.11 s**.
- If X is capped at the bottleneck ceiling, **extra users appear entirely as extra R**, not extra X.

6 Lost work — Erlang B

m = capacity: no. of slots (threads, connections, buffer)
B(m,ρ) = chance an arrival is dropped (1e-6 = 1 in a million)

Capacity m (threads, connections, lines, buffer slots), offered load ρ = λS (Erlangs):

$B(m, \rho) = (\rho^m/m!) / \Sigma_{k=0..m}\, \rho^k/k!$

Stable recursion: B(0) = 1, B(m) = ρ·B(m−1) / (m + ρ·B(m−1)). **Use when** a finite pool drops/rejects arrivals — find the smallest m with B(m, ρ) ≤ ε.

m	ρ=0.5	ρ=0.6	ρ=0.7	ρ=0.8	ρ=1.0
3	1.3e-2	2.0e-2	2.9e-2	3.9e-2	6.2e-2
4	1.6e-3	3.0e-3	5.0e-3	7.7e-3	1.5e-2
5	1.6e-4	3.6e-4	7.0e-4	1.2e-3	3.1e-3
6	1.3e-5	3.6e-5	8.1e-5	1.6e-4	5.1e-4
7	9.4e-7	3.0e-6	8.1e-6	1.9e-5	7.3e-5
8	5.9e-8	2.3e-7	7.1e-7	1.9e-6	9.1e-6

Example	Working	Answer
Router: 300 pkt/s in, 500 pkt/s capacity, drop < 10^{-6}	ρ = 0.6; B(7) = 3.0e-6 ×, B(8) = 2.3e-7 ✓	8 × 1,024 B = **8 KiB** buffer
DB pool: 7 req/s, S = 100 ms, ≥1 free conn 99.9%	N = 0.7; B(4,0.7) = 5.0e-3 ×, B(5,0.7) = 7.0e-4 ✓	**5 connections**

7 Service time before the system exists (execution models)

Use when the system is not built yet, so S cannot be measured. Annotate the design (UML activity / sequence diagram, Smith & Williams 2001) with times and probabilities, then *solve* for expected service time. Result feeds the bounds in §3 (D = V×S, X ≤ $1/D_{max}$, R ≥ ΣD) and sizing in §4.

7.1 METHOD (FIVE STEPS)

1. **Draw the flow** of one request: activities, messages, branches, parallel paths.
2. **Annotate** each activity with a time (ms) and its source (benchmark, vendor figure, similar system).
3. **Annotate** each branch with a probability (sum to 1) and each loop with an expected iteration count.
4. **Solve** inside-out with the rules below → expected time per request, E[S].
5. **Feed the bounds:** D = V×E[S] per device, ceilings c/D, then **validate by measurement** once something runs.

7.2 SOLVING RULES

t = time of a step · **E[t]** = expected (average) time · **p_i** = chance of branch i
k = no. of loop iterations · **p_fail** = chance one attempt fails

Structure	Rule	Formula	Notes
Sequence (A then B)	Add	$t = t_A + t_B$	Everything on the critical path
Parallel / fork-join	Longest path	$t = \max(t_A, t_B, \ldots)$	Join waits for the slowest branch
Async with overlap	Max of overlapped parts	$t = \max(t_{wait}, t_{work})$	Only if useful work really happens during the wait
Branch (p_i)	Weighted average	$E[t] = \Sigma\, p_i t_i$	$\Sigma\, p_i = 1$
Loop (k iterations)	Multiply	$t = k \cdot t_{body}$	Use the *mean* iteration count
Retry until success (general)	Expected attempts	$E[\text{attempts}] = 1/(1 - p_{fail})$	Each failed attempt adds its time (or a timeout)
Repeated device visits	Visits × time	D = V×S	Links the model to §3

ORDER OF SOLVING · Work **inside-out**: collapse each branch to its expected value, each parallel block to its maximum, then add the sequence. Common slip: **adding** parallel branches instead of taking the max.

7.3 WORKED EXAMPLES

Ex 1 — async call overlapped with branching work (L05). Decide 10 ms → fork: async network call 30 ms ‖ build response (50%×12, 40%×25, 10%×40 ms) → join → respond.
Build = 0.5(12) + 0.4(25) + 0.1(40) = 6 + 10 + 4 = **20 ms** → overlapped block max(30, 20) = **30 ms** → total 10 + 30 = **40 ms**. Not overlapped: 10 + 30 + 20 = 60 ms (overlap saves 20 ms).

Ex 2 — cache hit or miss (L05). Cache check 2 ms; 0.8 hit → done; 0.2 miss → search index +7 ms. **E[S] = 2 + 0.2 × 7 = 3.4 ms** (hit path 2 ms, miss path 9 ms). Bounds: X ≤ 1/0.0034 ≈ **294 req/s**; at 70% planning target ≈ **206 req/s**.

Ex 3 — MapReduce, serial + parallel phases (Tut Wk 5 Q3). Map (serial partitioning, 15 ms / 1,000 records) → Compute (parallel on n processors, 0.1 ms/record) → Reduce (serial, 1 ms/partition):

S(n, r) = 0.015r (Map) **+ 0.1r/n** (Compute) **+ n** (Reduce) **[ms]**

Case (r = 100,000)	Map	Compute	Reduce	S
n = 1	1,500	10,000	1	11,501 ms
n = 8	1,500	12,500 × 0.1 = 1,250	8	**2,758 ms**
n = 16	1,500	625	16	2,141 ms
n = 100 (optimum)	1,500	100	100	**1,700 ms**
n = 200	1,500	50	200	1,750 ms (worse)

- **Optimal n:** dS/dn = −0.1r/n² + 1 = 0 ⇒ **n* = √(0.1r)** = √10,000 = **100**.
- **Floor:** serial Map (1,500 ms) never parallelises away; 8 → 100 processors (12.5× hardware) cuts S only 1.62×.
- **Amdahl (general):** serial fraction ≈ 1,500/11,501 = 13% → max speedup ≈ 7.7×; Reduce makes very large n *worse*.

Ex 4 — parallel fan-out. Gateway 5 ms, three backends **in parallel** (20, 35, 50 ms), then merge.

Design	Working	S
Parallel calls	5 + max(20, 35, 50)	**55 ms**
Sequential calls	5 + 20 + 35 + 50	110 ms

Slowest backend sets the time: speeding the 20 ms call changes nothing; the 50 ms call helps until < 35 ms — the *critical path*.

Ex 5 — agent loop with retries (illustrative, Wk 9–10). Each step: LLM call 800 ms, then two tool calls in parallel (300, 500 ms). Each step fails and retries with p = 0.1 (failed attempt costs the same). Task = 3 steps.

Quantity	Working	Result
One attempt	800 + max(300, 500)	1,300 ms
Expected attempts	1 / (1 − 0.1)	1.11
Expected time per step	1,300 × 1.11	≈ 1,444 ms
Whole task	3 × 1,444	**≈ 4.33 s** (vs 3.9 s, no retries)

7 Execution models (cont.)

7.4 FROM MEAN TO VARIABILITY (TIES TO KINGMAN)

The expected value hides spread; the same branch probabilities give C_s for free:

$$\mathbf{Var(t) = \Sigma\, p_i(t_i - E[t])^2 \cdot C_s = \sqrt{Var(t)} / E[t]}$$

Model	E[t]	Var	C_s	C_s^2
Ex 1 build step (12 / 25 / 40 ms)	20 ms	82	0.45	0.20
Ex 2 cache (2 ms or 9 ms)	3.4 ms	7.84	0.82	0.68

Plug C_s^2 into Kingman to estimate queueing delay *before* building. A rare, very slow branch (report query, miss to a slow store) raises C_s sharply even when the mean looks harmless.

7.5 COMMON MISTAKES

Mistake	Fix
Adding parallel branches	Take the **maximum**
Branch probabilities do not sum to 1	Check $\Sigma\, p_i = 1$ before solving
Assuming async always overlaps	Only if there is useful work during the wait
Using E[S] as a latency guarantee	It is a **mean**; check C_s and the tail
Forgetting repeated visits	Multiply by V to get demand D
Trusting estimates	Validate once measurable. Demo: three S for one service (112 ms sequential, 35 ms inferred, 70 ms at ceiling) — record how each was obtained

7.6 PRACTICE (ANSWERS IN BOLD)

1. 5 ms setup, then 70% → 4 ms path, 30% → 20 ms path. E[S] and ceiling? **5 + 2.8 + 6 = 13.8 ms; 1/0.0138 ≈ 72 req/s**
2. Three parallel tasks (12, 18, 9 ms), then 4 ms merge. S? **max(12,18,9) + 4 = 22 ms**
3. Loop of mean 3 iterations × 6 ms, then 2 ms cleanup. S? **3 × 6 + 2 = 20 ms**
4. MapReduce, r = 50,000, n = 4 (Ex 3 settings). S and n*? **750 + 1,250 + 4 = 2,004 ms; n* = √5,000 ≈ 71**

8 Performance testing

8.1 WHICH TEST DO I NEED?

Meets target at expected load? → **Load** · Where/how does it break? → **Stress** · Healthy over hours? → **Soak** · Did the last change slow it? → **Regression** · A vs B vs standard? → **Benchmark** · Can I trust the test? → **Pipe-clean**

Test	Varies	Look for	Use when…	Course example
Load	Nothing (hold λ)	Flat R, X = λ, errors ≈ 0; decide vs written requirement	Sign-off against target	10 req/s × 3 min: p95 120 ms, 0% err, CPU 16% vs predicted 15%
Stress	Load ↑ in steps	**Knee** (p95 climbs), **ceiling** (X flat), **failure mode**, recovery	Estimate may be wrong; need limits, graceful degradation	5→30 req/s, 1 core: knee 5–10, ceiling ≈ 13–15, 50% err at 30
Soak	Time	Memory **slope**, conn/file-handle growth, R drift	Suspect leaks; long-running service	3.4 MB/min straight line, no plateau → leak; 16 GB in 3.3 days
Regression	Build	p95 and X vs baseline; one change per build	Every merge in CI	Duplicate-check scan: p50 +30%, p95 +45%
Benchmark	Candidate	Side-by-side table; one variable at a time	Choosing configs/models/vendors (TPC, MLPerf, SPEC)	PBKDF2 25k/100k/400k: p95 110 / 480 ms / collapse (83% timeouts)
Pipe-clean	Nothing (low load)	Assertions that can't fail; counts reconcile	**Before every real run**	Broken path, no body assertion: 100% "pass", 0 orders
Isolation	One component	Which tier owns the time	Full test shows a problem	Time PBKDF2 alone in a loop
UAT	Real users	Does it *feel* acceptable	Validating the targets	1.9 s vs "2 s" target, users still complain

Pokémon Go (2016): planned 5×, got 50×. A **load test** at 5× passes and is silent about 50×. A **stress test** finds the ceiling *and the ungraceful failure mode* → admission control / waiting room.

8.2 OPEN LOOP VS CLOSED LOOP

	Closed loop	Open loop
Behaviour	Each user waits for reply, thinks, repeats	Arrivals at fixed λ regardless of replies
Offered load	N/(R+Z) — **backs off** when system slows	λ — queue grows if system falls behind
JMeter	Standard Thread Group	Open Model Thread Group / Throughput Shaping Timer
Other tools	Locust (default)	k6 constant-arrival-rate
Use for	Modelling humans in sessions	Real web traffic; **any result compared with a queueing model**

WARNING · A closed-loop result is not evidence of throughput capacity — it hides saturation by sending less load.

8.3 READING A RESULTS ROW

Column	Check
offered	Count λ from the `.jtl`, not the requested rate
samples	Excludes warm-up; **must equal** server-log lines
p50/p95/p99	Compare to requirement *and to prediction*
err %	Defined by **body assertion**, not HTTP status
throughput	= offered ⇒ kept up; < offered ⇒ queueing/failure
cpu_pct	Compare with λS/c (whole machine: one core of 8 ≈ 12.5%)
rss_mb[app]	Constant at constant load; upward line = leak

Prediction check (Demo 1): U = λS/c = 10 × 0.034 / 1 = **0.34** → measured CPU 35–37%. When prediction and measurement disagree, *the gap is where the work is.*

Ceiling estimates: X_{max} = c/S. Sequential S = 112 ms → 9/s; inferred 35 ms → 28/s; actual ≈ 70 ms → 14/s. Record how each S was obtained.

8.4 TESTABLE REQUIREMENT (BONDI, 10 FIELDS)

1 Requirement number	2 Title	**3 Statement**	4 Supporting commentary	5 Precedents / sources
6 Derivation of quantities	7 Dependent requirements	**8 Assumptions**	**9 Measurement**	10 Subject-matter expert

Fields **3, 6, 8, 9** are where requirements usually fail.

Template: *"In [x]% of cases, under [load], the [operation] shall have a [statistic] response time of no more than [n] [unit]."* e.g. *95% of POST /orders complete within 200 ms at 10 req/s (open loop) for 10 minutes on reference hardware.*

Seven quality checks: unambiguous · measurable · verifiable · complete · consistent (5 req/s × 5 s response needs ≥ 25 in flight by Little — impossible with 2 workers) · traceable · time scale stated.

Four documents: requirements (everyone, the contract) → measurement plan (instruments) → test plan (tests, playbook) → capacity management plan (operations).

SATURATED vs CONGESTED (goes with §3)
Saturated = work arrives faster than the slowest station can handle it.
Test: arrival rate ≥ the lowest ceiling, i.e. some station is busy 100% (U ≥ 1).
Result: the queue grows forever and output is stuck at the ceiling. Fix: add capacity.
Congested = long queue, but every station is below 100% busy.
Cause: bursty arrivals / uneven service (variability, Kingman). Fix: cut variability.
Fab example: arrivals 3.05/day; slowest station = deposition at 24/6.7 = 3.6/day. 3.05 is below 3.6, so NOT saturated, even though the litho queue tripled (congested). If 2 litho tools fail: litho ceiling = 24/8.6 = 2.8/day, below 3.05, so it SATURATES.

8.5 TEST ENVIRONMENT AND PLAYBOOK

Rule	Why
Reproducible, controlled, recorded	Someone else must get the same answer; keep raw `.jtl` and server log
≥ 3 runs per configuration	One run is a number; three show whether it means anything
Discard warm-up; state window start	Report steady state only
Generator on a different machine	Else it competes for CPU and removes network delay
Measure the cost of measuring	Profilers/loggers are load; `cProfile` can double R in pure-Python handlers
Match production in **structure**	Same config, data shape/size, state, dependencies (stubs), network hops
Scale **all** components together	Ceiling scales with cores; soak lifetime with RAM — not together
Write down what it doesn't cover	That list is a result

Playbook: A Initialise (log commit hash, record settings, empty store, start monitor) → **B** Run (**B1** one request by hand; run plan; don't touch) → **C** Record (copy logs, **C3 reconcile** samples = server log = stored orders). Counts differ ⇒ discard the run.

8.6 LOAD MODEL

User types	One script per behaviour; mix matches the workload model
Think time	Zero turns users into tight loops. Constant Timer = fixed pause; **Gaussian Random Timer** = varies around a mean
Pacing	Big bang (start-up transient) · ramp-up (steady state) · ramp-down (recovery)
Load profile	Flat · stepped (stress) · spike · replayed trace
Data	CSV Data Set Config, realistic in size — not the same order 3,000 times

JMeter non-GUI: `jmeter -n -t plan.jmx -l results.csv -e -o report/` (validate in GUI first).

8.7 TOOLS

JMeter — Java, GUI to build, CLI to run; Open Model Thread Group · **k6** — JavaScript, CI-friendly, open-loop executors · **Locust** — Python, closed-loop by default · **LoadRunner** — commercial, large enterprises.

9 Optimisation

h = cache hit rate · **S_hit / S_miss** = time on a hit / miss · **Seff** = average S
p = fraction of work sped up · **s** = how many times faster it gets

9.1 SYMPTOM → CAUSE → FIX (FOUR SOURCES)

Symptom	Source	Law	Fix
High U on one device; X plateaus	**Capacity**	X ≤ c/D	Add servers/replicas, faster hardware
Bottleneck busy as each txn asks too much	**Demand**	D = V×S	Cache (↓V), index/algorithm (↓S)
High p95/p99 at moderate U	**Variability**	Kingman	Split heavy/light queues; jitter retries; smooth bursts
R grows with users, X flat	**Population**	R = N/X – Z	Raise ceiling, cap N, admission control

9.2 TECHNIQUE REFERENCE

Technique	Mechanism	Formula / effect	Use when…
Indexing	Lookup avoids scans	O(n) → O(log n)	Scans dominate (O(n) duplicate check)
Algorithmic	Better complexity	$O(n^2)$ → O(n log n)	Profiler shows hot loop; AI-generated code
Caching	Reuse results	$S_{eff} = h \cdot S_{hit} + (1-h) \cdot S_{miss}$; ↓V at device	Repeated reads; DB is bottleneck
Concurrency	Parallel execution	X rises until a resource saturates, then falls (contention)	Idle capacity elsewhere
Separate queues	Isolate heavy from light	C_s^2 collapses; Kingman W falls	Mixed workloads (reports vs standard)
Replicas	More servers	Ceiling c/D ↑; ρ = λS/c ↓	Capacity-bound
Admission control	Reject early, bound queue	Stops wasted work on abandoned requests	Stress test showed ungraceful collapse
Headroom	Run below 70–80% U	Keeps you left of the knee	Interactive (uniform batch can run hotter)
Jittered retries	Spread bursts	↓ C_a	Retry storms, on-the-hour jobs
LLM serving	Continuous batching; split prefill/decode	↓ C_s (token counts vary)	Inference servers

Cache arithmetic: hit 2 ms, miss 50 ms → 90% hit ⇒ 6.8 ms; 99% ⇒ **2.48 ms** (2.7× better from 9 more points of hit rate).

Amdahl (general): fraction p improved by factor s → **Speedup = 1 / ((1–p) + p/s)**; limit s → ∞: **1/(1–p)**. p = 0.6, s = 4 → 1.82×; ceiling 2.5× however fast the part becomes.

Levers on D = V×S: halve V (4 → 2 queries) ⇒ halve D ⇒ ceiling doubles at that device. **Iterative:** fix bottleneck → recompute all ceilings → next bottleneck appears.

9.3 VERIFIED RESULTS FROM THE COURSE

Case	Before	After	Why
Online service	p95 **3.48 s**, DB $C_s^2 \approx 34$	p95 **0.44 s** (8×), DB wait 165 → 5 ms	Reports → read replica (variance, not load)
Kingman on DB-RPT	Predicted 0.30 s	Measured 0.27 s (within 8%)	Valid where assumptions hold; over-predicts 2–3× on shared DB-01
Wafer fab	Cycle 1.8 → 2.2 days	Throughput unchanged	Lost tool: ρ 0.36 → 0.54, $C_a \approx 1.5$ amplified wait
PBKDF2 benchmark	400k iterations	83% timeouts	Hashed for clients already gone; X fell *below* ceiling

10 Exam checklist

1. **Name the source:** Demand / Capacity / Variability / Population.
2. **Draw the system:** devices, visit counts V, server counts c.
3. **Compute demands** $D_k = V_k S_k$, then **ceilings** c_k/D_k; bottleneck = lowest.
4. **Pick the law:** forced flow (device load) · Little (counts) · interactive (closed) · Kingman (tail at moderate U) · Erlang B (drops).
5. **Sanity-check:** U < 1; R ≥ S; R ≥ ΣD; units on every line.

COMMON TRAPS

- Using **λ** in U = λS when overloaded — use U = XS.
- Choosing the **largest D** instead of the **lowest c/D** ceiling.
- Forgetting to divide by **c** (servers).
- λ = 1 / mean observed gap instead of arrivals ÷ whole window.
- Trusting **mean** latency; asserting only HTTP **status** (200 with error body passes).
- Falling p50 under load read as good — fast failures drag the median down; check errors.
- Comparing a **closed-loop** result with a queueing-model prediction.
- Reporting soak growth instead of **slope**, or one run instead of three.
- Forgetting the **cap** at total users N in JMeter concurrency.

11 Symbols

λ	Arrival rate (demand)	**N**	Number in system / virtual users
μ	Service rate = 1/S	**Z**	Think time
S	Service time	$\mathbf{V_k}$	Visits to device k
W	Wait (queue) time	$\mathbf{D_k}$	Service demand = $V_k S_k$
R	Response time = W + S	**c**	Number of servers
X	Throughput (supply)	$\mathbf{C_a}$, $\mathbf{C_s}$	CV of inter-arrival / service times
U	Utilisation	**ρ**	Traffic intensity (λS, or λS/c)
B(m,ρ)	Erlang B loss probability	**L**	Concurrent users

12 Profiling toolkit (Lab 1)

Use when: you must find *where* one program spends time or memory before optimising. The profile gives the service time S that §1 and §4 multiply by λ.

	Deterministic	Statistical
How	Hooks every call/return	Samples the stack N×/s
Overhead	High (2–10×), distorts	Low; safe on production
Output	Exact calls and times	Sample share ≈ time share
Tools	`cProfile`	`py-spy` , `perf`
Use when	Offline; need call counts	Long-running process you cannot restart

WHICH TOOL?

Question	Tool · call
Time one block	`perf_counter()` before/after
Repeatable snippet timing	`timeit.repeat(stmt, setup, repeat=10, number=1)` → report **min**
Which function is hot?	`python -m cProfile -s tottime s.py`
Call tree picture	`gprof2dot` / `snakeviz` on `.pstats`
Live or production process	`py-spy record --rate 100 --format flamegraph -o f.svg -- python s.py` · `py-spy top --pid N` · `py-spy dump --pid N`
Which *line* grows memory?	`memory_profiler` : `@profile` , `%memit`
Which allocation site; peak?	`tracemalloc` : `take_snapshot().compare_to()` , `get_traced_memory()` → (current, peak)
Which machine?	`psutil` , `platform` : record with every result

READING `CPROFILE`

Column	Meaning
ncalls	Calls; `36/1` = 36 total, 1 primitive (recursion)
tottime	Self time, excluding callees → **optimise here**
cumtime	Self plus everything called → find expensive trees

Flame graph: width = share of samples; x order is alphabetical, **not** time; y = stack depth.

Formula	Use	Example
$\mathbf{t_{prof} / t_{plain}}$	Profiler overhead	1.83 s → 9.56 s = **5.2×**
$\mathbf{f_i = n_i / \Sigma n}$	`py-spy` leaf share	func_b 49%, func_c 32%, func_e 18%
1 / ((1−p) + p/s)	Amdahl (general)	2× on a 1% function → **1.005×**
T(n) = T(n−1)+T(n−2)+1	Naive recursion O(φⁿ); memoised O(n)	`fib(30)` : **2,692,537** calls vs `fib(34)` memoised: **35**

MEASURED RESULTS

Test	Result
Insertion ÷ merge sort, n = 500 / 1k / 2k / 4k	×6.7 / ×12.6 / ×24.7 / ×50.4 (grows ≈ n / log n)
`tracemalloc` peak, n = 2000	insertion 0.8 KiB · quick 1.7 KiB · merge 19.5 KiB (copies)
Local versus Colab (2-core Xeon)	9.56 s versus 12.79 s: **times do not transfer; call counts do**

TRAPS

- `timeit` **setup runs once**: an in-place sort sees sorted data from run 2. Use `number=1` with `repeat` ; report the **minimum**, not the mean.
- Take an **unprofiled baseline** first; the profiler distorts what it measures.
- Fix the RNG seed; record the environment (`psutil`).
- `memory_profiler` samples RSS (interpreter baseline ≈ 64–125 MiB) and is unreadable on recursion → use `tracemalloc` for peaks.
- `py-spy top` is a terminal UI → capture `--format raw` .
- No Graphviz `dot` on Windows → plot `.pstats` in matplotlib.
- **Final answer right ≠ cache right.** The handout `fib` recursed on n−1 instead of n−2, shifting every cached index; check `fib_dict[10]` = 55. A profiler cannot catch it.

METHOD · Record environment → fix inputs → unprofiled baseline → profile → optimise the top `tottime` → re-measure → repeat until flat or on target.

13 Flask + JMeter lab (Lab 2)

Flask `/sort` endpoint, `flask-profiler` (sqlite, dashboard `/flask-profiler/`), JMeter 5.6.

Item	Rule
Build order	Validate in GUI (Thread Group → HTTP Request → Results Tree), then run **non-GUI**
Command	`jmeter -n -t plan.jmx -Jthreads=50 -Jrampup=5 -Jloops=10 -l results.csv -e -o report/`
Parametrise	`${__P(threads,1)}` so one plan serves every run
Arrival rate	λ = threads / ramp-up: 50/5 = **10/s**; 5000/50 = 100/s
Samples	threads × loops × samplers: 50 × 10 = **500**
Concurrency	L = λW (Little), capped at N
Knee	Plot R against active threads; lab: R < 50 ms up to ≈ **17 users**
Stress recipe	5000 threads / 50 s ramp-up: instructor CPU only, a recipe not a result
Think time	Constant Timer (fixed) or Gaussian Random Timer (offset ± deviation)

THE "RECURSIVE USE OF CURSORS" CRASH

Step	Finding
Symptom	50 threads / 5 s / 10 loops: **100% errors**, `sqlite3.ProgrammingError: Recursive use of cursors not allowed`
Root cause	Flask `app.run()` is **threaded by default**; flask-profiler's sqlite backend shares **one cursor**. `check_same_thread=False` lifts the guard but does not make concurrent `execute()` safe
Control run	`threaded=False` : **0% errors**, 500/500, avg 158 ms
Fixes	`threaded=False` (loses concurrency) · a `threading.Lock` · MongoDB storage (`pymongo==3.13.0` , pooled connections)

METHOD · Isolate the cause by changing **one variable** (the threading model). Do not blame the load tool or the network without a control run.

LAB LESSONS

- **The profiler is an observer:** it writes every call to storage and can cap the system's throughput. Its time is wall-clock (running *plus* I/O and GIL waiting).
- **View Results Tree** stores every sample: remove it; use non-GUI plus the HTML dashboard.
- **Tester and server on one machine** compete for CPU; separate them (Docker Compose).
- **Close the profiler dashboard** during a test: each load fires ~10 asset/API calls and a 308 redirect.
- **Read the server log** before blaming the endpoint (one 404 was a URL pasted after the Colab host).
- `curl` needs `-g` (or quotes) for URLs containing `[ ]` .
- Venv: `python -m venv test_env` → activate → `pip install flask flask-profiler==1.8.1` .
- **Apdex = 0.000** when every response exceeds T (`time.sleep(5)` against T = 500 ms).

14 Assignment 1: ticket triage

Test an intentionally naive, CPU-only LLM ticket classifier (7 categories) to get an honest baseline.

1. **Golden set first** (150–200 items): label before any model exists; drop the source label so raters are not anchored; resolve disagreements.
2. **Naive baseline:** synchronous `POST /tickets` , no cache or queue, fresh SQLite connection per request. Do **not** pre-optimise.
3. **Instrument:** one JSON log line per request (`ticket_id` , `model` , `category` , `classify_latency_ms` , `fallback_used`); every JMeter figure must reconcile with the log.
4. **Swap model only via** `OLLAMA_MODEL` : 3–5 pinned models in ≥ 2 size classes, identical harness.
5. **Apply §8:** load (expected λ) · stress (find the knee: Ollama CPU, SQLite locks) · pipe-clean (body category ∈ the 7) · benchmark per model, one variable at a time.

Fallback: exact → substring → fixed category. `fallback_used` separates "model wrong" from "ignored the output format".

Quantity	Formula	Example
Accuracy · fallback rate (general)	correct ÷ total · fallback ÷ total	—
Cohen's κ (2 raters)	$(p_o - p_e) / (1 - p_e)$	p_o = 0.80, 3 equal categories → p_e = ⅓ → κ = **0.70**
Fleiss' κ (3+ raters)	$(\bar{P} - \bar{P}_e) / (1 - \bar{P}_e)$	Averaged over items
Landis & Koch (general)	<0 poor · 0–.20 slight · .21–.40 fair · .41–.60 moderate · **.61–.80 substantial** · .81–1 almost perfect	

Tips: read the CSV with `csv.DictReader` (narratives contain newlines, so `wc -l` overcounts); use `with TestClient(app)` so start-up events run; mock `classify` when Ollama or Docker is unavailable.

15 Capacity-planning formulas (Tutorial Wk 4)

Quantity	Formula	Example
End-to-end yield	Y = Π pass rates	0.90 × 0.975 = **87.75%**
Required incoming	target / Y	100,000 / (0.96 × 0.95) = **109,650**/day (round up)
Mixed demand	$D = \Sigma\, p_i D_i$	0.9(5) + 0.1(30) = **7.5 ms**; 120/s → 90%
Mean fan-out	$V = \Sigma\, p_i V_i$	0.9(2) + 0.1(8) = 2.6; × 1,200 = **3,120 qps**
Retry visits	$V = 1/(1-p_{fail})$; attempts/s = X·V	p = 0.1 → 1.111; 90 → **100**/s; × 8 ms = 80%
Instances needed	$m = \lceil \lambda S / U_{cap} \rceil$	250 × 0.04 = 10; 10 / 0.8 = 12.5 → **13** (76.9%)
Max rate from bottleneck	$X_{max} = c/S$, × downstream pass rate	24 × 25 = 600 × 0.96 = **576** orders/s
Propagate upstream	$\lambda_{up} = \lambda_{down}$ / pass rate	600 / 0.8 = 750/s; 250 × 2 ms = **50%**
Thread-pool check	$\bar{n} = \lambda R$ vs thread count (mean, not peak)	250 × 0.12 = 30 < 48; saturates at R = 48/250 = **192 ms**
SLO tail count	(1 − p) × N	5% × 1,080,000 = **54,000** above p95
Growth to saturation	$t = \ln(1/U_0) / \ln(1+g)$	U_0 = 0.8, g = 20% → **1.22 yr**
Query optimisation	$X_{max} = U_{cap} / D_{new}$	0.8 / 1.25 ms = 640 < 691 → buys ≈ **2.6 yr**
Read replicas	$D_{pri} = (1-r)D + rD/(1+k)$	r = 0.7, k = 2: 0.6 + 0.467 = 1.067 ms → U = **73.7%**

PATTERN · Find the bottleneck → compute U at the planning horizon → compare each option against the U cap (70–80%).